

InternSight — Full Platform Context Document

Version 1.0 | For AI context injection and founder reference

Stack: Next.js · Convex · FastAPI · Groq AI · Gemini · Stripe · Clerk

1. What InternSight Is

InternSight is a **premium AI-powered internship success platform** for CS and engineering students. It is not a resume feedback tool. It is not a job board. It is a **decision and optimization system** that answers one question for students:

"Am I ready for this role? If not, exactly what do I fix?"

The platform turns a student's resume into structured data, scores it against real internship role expectations, surfaces roles they can actually win, and guides them through fixing gaps and applying strategically — all in one place.

Core promise to users: Confidence you will get interviews, not just "feedback."

2. Pricing Model

Tier	Price	Access
Free	\$0	1 resume upload + score only (no rewrite, no role alignment, no copilot)
Pro	\$5.99/month	Unlimited everything — all 6 engines, unlimited uploads, full copilot

Stripe implementation:

- Payment processor: Stripe
- Product: one flat subscription at \$5.99/month (no annual tier yet)
- Checkout: Stripe Checkout Session (redirect model — simplest, most reliable)
- Webhook: `customer.subscription.created` and `customer.subscription.deleted` → update `user.isPro` in Convex
- Gate enforcement: check `user.isPro` in FastAPI middleware AND in Next.js route guards
- Stripe env vars needed: `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PRICE_ID`

What's gated (Pro only):

- Resume rewriting (Jake-style)
- Role Alignment Engine
- Internship Discovery fit scoring
- AI Copilot
- Application Strategy Engine
- Resume Builder (live scoring + suggestions)
- Multiple resume versions

What's free:

- Resume upload + parse
 - Basic score (0-100) + breakdown
 - Weakness detection list (read-only, no suggestions)
 - Static internship listings (no fit scoring)
-

3. Tech Stack

Frontend

- **Next.js 14** (App Router) — all pages and routing
- **Tailwind CSS** — all styling
- **shadcn/ui** — component library (buttons, dialogs, cards, etc.)
- **Framer Motion** — page transitions and micro-animations
- **react-dropzone** — resume file upload
- **@dnd-kit/core** — kanban drag-and-drop in application tracker
- **react-pdf** — PDF generation for resume export

Auth

- **Clerk** — authentication, session management, user metadata
- Clerk webhook syncs new users to Convex `users` table on signup

App Backend

- **Convex** — primary database, file storage (resumes), real-time sync, scheduled functions

- **Convex handles:** user data, resume storage, analysis results, application tracker, internship cache, subscription status
- **Real-time subscriptions power:** live resume score updates, tracker kanban sync

AI Backend

- **FastAPI** (Python) — all AI processing, scoring, and inference
- Deployed on: Railway (or Render)
- Exposes REST API consumed by Next.js server actions

AI Models (see Section 5 for routing logic)

- **Groq AI** — primary inference engine (llama-3.3-70b-versatile, llama-3.1-8b-instant)
- **Google Gemini** — fallback / load balancer (gemini-1.5-flash)
- Both accessed via their respective Python SDKs from FastAPI

Payments

- **Stripe** — subscription billing
- Stripe Checkout for payment flow
- Stripe webhooks → FastAPI endpoint → Convex mutation

Deployment

- **Vercel** — Next.js frontend
- **Railway** — FastAPI AI backend
- **Convex Cloud** — database and backend functions

Monitoring

- **Sentry** — error tracking (frontend + FastAPI)
- **Posthog** — product analytics

4. The Six Core Engines

Engine 1: Resume Intelligence Engine (The Core Brain)

Location: FastAPI `/api/resume/`

Status: Primary value engine. Users pay for this first.

What it does:

1. Accepts PDF or DOCX resume upload
2. Extracts raw text (pdfplumber for PDF, python-docx for DOCX)
3. Sends extracted text to AI model with structured extraction prompt
4. Parses response into structured resume object
5. Scores the resume across 4 dimensions (0-25 each = 100 total)
6. Detects specific weaknesses with severity and suggestions
7. Rewrites the entire resume in Jake's Resume format

Scoring breakdown:

- Experience Quality (0-25): impact verbs, relevance to target role, description depth
- Metrics & Impact (0-25): % of bullets with quantified results, numbers present
- Structure (0-25): required sections present, Jake-format adherence, ATS compatibility
- Keywords (0-25): ATS keyword density matched to target role

Weakness types detected:

- `MISSING_METRICS` — bullet has no numbers or percentages
- `WEAK_VERB` — bullet starts with passive/weak verb (helped, assisted, worked on)
- `MISSING_SECTION` — no summary, skills, or projects section
- `VAGUE_DESCRIPTION` — description gives no concrete output or result
- `ATS_RISK` — formatting that ATS systems may fail to parse (tables, columns, headers)

Jake's Resume format rules enforced in rewrite:

- Single column, clean serif/sans fonts
- Sections: Name/Contact → Summary (optional) → Education → Experience → Projects → Skills
- Each experience bullet: Action Verb + Task + Result + Metric
- No photos, no colors, no tables, no headers/footers
- 1 page for students, 2 pages max for grad students

FastAPI endpoints:

- `POST /api/resume/parse` — extract + structure
- `POST /api/resume/score` — score + weaknesses

- `POST /api/resume/rewrite` — full Jake-style rewrite
 - `POST /api/resume/analyze` — combined pipeline (parse + score + rewrite in one call)
-

Engine 2: Role Alignment Engine (The Differentiator)

Location: FastAPI `/api/roles/`

Status: Key product differentiator. Honest truth about readiness.

What it does:

1. User selects target role(s): SWE Intern, ML Intern, Data Eng, DevOps, Product, Research
2. System compares parsed resume against role expectations schema
3. AI evaluates competitiveness with honest categorization
4. Returns specific gaps and what to fix

Role expectation schema (example — SWE Intern):

```
json
{
  "role": "swe_intern",
  "label": "Software Engineering Intern",
  "required_skills": ["data structures", "algorithms", "one OOP language", "git"],
  "nice_to_have": ["system design basics", "web frameworks", "databases", "API design"],
  "project_depth": "at least 1 project with backend or fullstack component",
  "experience_min": "any technical experience or strong projects",
  "gpa_soft_threshold": 3.0,
  "target_companies": ["Google", "Meta", "Apple", "Netflix", "Stripe", "startups"]
}
```

Competitiveness levels:

- `READY` — resume meets or exceeds role expectations, apply now
- `CLOSE` — 1-2 specific gaps, fixable in 2-4 weeks
- `NOT_READY` — multiple fundamental gaps, fix before applying

Output per role:

- Competitiveness level + reasoning
- Match percentage (skills overlap)

- Top 3 specific gaps: e.g., "Missing: no async/concurrent programming demonstrated"
- "What to fix" action list: concrete next steps
- Estimated time to readiness (for CLOSE/NOT_READY)

FastAPI endpoints:

- `GET /api/roles/` — list all available roles with descriptions
 - `POST /api/roles/align` — evaluate resume against one or more roles
 - `GET /api/roles/{role_id}` — full role expectation schema
-

Engine 3: Internship Discovery Engine

Location: FastAPI `/api/internships/` + Convex scheduled functions

Status: Turns the app into a strategy tool.

What it does:

1. Pulls internship listings from external APIs (Adzuna, RapidAPI LinkedIn Jobs)
2. Caches listings in Convex with daily refresh (Convex scheduled function)
3. Scores each listing against the user's parsed resume
4. Categorizes listings into three swimlanes
5. Generates AI explanations for fit/no-fit

Listing schema:

```
typescript
{
  id: string,
  title: string,
  company: string,
  location: string,
  remote: boolean,
  skills_required: string[],
  level: "intern" | "entry",
  url: string,
  posted_at: number, // timestamp
  source: "adzuna" | "linkedin" | "greenhouse"
}
```

Swimlane categories:

- `APPLY_NOW` — match score > 70%, apply immediately
- `STRETCH` — match score 40-70%, apply but improve specific things first
- `NOT_READY` — match score < 40%, shown as "build these skills first"

Per-listing output:

- Match score (%)
- "Why you fit" — 2-3 sentences from AI
- "Why you don't fit" — specific missing skills
- "Improve before applying" — 1-2 action items

Data sources:

- Primary: Adzuna Jobs API (generous free tier)
- Fallback: RapidAPI LinkedIn Jobs scraper
- Future: direct Greenhouse/Lever API integrations

FastAPI endpoints:

- `GET /api/internships/` — paginated listing browse (no auth needed)
- `POST /api/internships/match` — score listings against user's resume

Engine 4: Application Strategy Engine

Location: Convex + FastAPI `/api/strategy/`

Status: Premium retention layer. Justifies subscription.

What it does:

1. Tracks every application the user makes
2. Monitors application velocity and response rates
3. Generates AI strategy recommendations weekly
4. Sends follow-up reminders at 7-day intervals

Kanban columns:

- Saved → Applied → Phone Screen → Interview → Offer → Rejected

Application card schema:

typescript

```
{
  id: string,
  userId: string,
  company: string,
  role: string,
  url: string,
  appliedAt: number,
  status: KanbanColumn,
  notes: string,
  followUpSentAt?: number,
  listingMatchScore?: number
}
```

Strategy intelligence outputs:

- Weekly summary: "You applied to 8 roles, 0 ML positions — you're missing your target"
- Pattern detection: "0/6 FAANG applications resulted in callbacks — focus mid-size first"
- Alerts: role closing soon, 7-day follow-up due, application velocity drop
- Application velocity chart (applications per week over time)

Follow-up system:

- Convex `crons` scheduled function runs daily
- For each application where `appliedAt` was 7 days ago and no follow-up sent: trigger
- Reminder sent via Resend email API
- Claude/Groq generates personalized follow-up email template

FastAPI endpoints:

- `POST /api/strategy/weekly` — generate weekly strategy report
- `POST /api/strategy/followup-template` — generate follow-up email for a specific application

Engine 5: AI Copilot

Location: FastAPI `/api/copilot/` (streaming SSE)

Status: Makes the product feel alive. Premium differentiator.

What it does:

1. ChatGPT-like interface inside the app
2. Has full context: parsed resume, score breakdown, target role, alignment results
3. Answers questions about the user's specific situation
4. Streams responses token by token

System prompt context injected:

```
python
```

```
system_prompt = f"""
You are InternSight's AI career advisor. You have access to this student's complete

RESUME SCORE: {score}/100
- Experience Quality: {breakdown.experience}/25
- Metrics & Impact: {breakdown.metrics}/25
- Structure: {breakdown.structure}/25
- Keywords: {breakdown.keywords}/25

TARGET ROLE: {user.targetRole}
ROLE READINESS: {alignment.status} ({alignment.matchPct}% match)
TOP GAPS: {'', '.join(alignment.gaps[:3])}

PARSED RESUME SUMMARY:
{resume_summary}

Your job is to give specific, actionable advice based on THIS student's actual resume.
Never give generic advice. Always reference their specific experience, scores, and gaps.
Be honest and direct. Students want truth, not encouragement.
"""
```

Suggested prompts shown to user:

- "Am I ready for a Google SWE internship?"
- "What's the single most important thing I should fix?"
- "How do I improve this bullet point: [paste it]"
- "What projects should I build to get into ML roles?"
- "Is this resume good enough to apply to Stripe?"

FastAPI endpoints:

- `POST /api/copilot/chat` — streaming SSE endpoint, accepts `messages[]` array
-

Engine 6: Resume Builder

Location: Next.js (client-side) + Convex (persistence) + FastAPI (live scoring)

Status: Daily-use sticky feature.

What it does:

1. Section-based editor (not a raw text editor)
2. Enforces Jake-style structure in real time
3. Shows live score updates as user types
4. Suggests better bullet phrasing inline
5. Exports to PDF in perfect Jake format

Editor sections:

- Header (name, email, phone, LinkedIn, GitHub)
- Education (school, degree, GPA, graduation date, relevant coursework)
- Experience (company, title, dates, bullets[])
- Projects (name, tech stack, dates, bullets[])
- Skills (languages, frameworks, tools — categorized)

Live intelligence:

- Score recalculates 500ms after each keystroke (debounced)
- Weak bullets highlighted in amber, strong in green
- Inline tip shown on focus: "This bullet has no metric — add a % or number"
- Word count warning on bullets > 20 words

Export:

- react-pdf renders exact Jake format
 - Downloadable as `firstname_lastname_resume.pdf`
 - Fonts: LaTeX Computer Modern or Source Serif
-

5. AI Routing Architecture

InternSight uses a **two-tier AI routing system** with Groq as primary and Gemini as fallback. No Claude or OpenAI — kept cost at zero during development.

Models in use

Provider	Model	Use case	Free tier
Groq	llama-3.3-70b-versatile	Resume rewrite, role alignment, copilot	14,400 req/day, 6k tokens/min
Groq	llama-3.1-8b-instant	Scoring, weakness detection (fast tasks)	14,400 req/day, 30k tokens/min
Gemini	gemini-1.5-flash	Fallback for all tasks when Groq rate-limited	1,500 req/day, 1M tokens/day

Routing logic (FastAPI)

```
python
```

```

# ai_router.py
import groq
import google.generativeai as genai
from tenacity import retry, stop_after_attempt, wait_exponential

groq_client = groq.AsyncGroq(api_key=os.environ["GROQ_API_KEY"])
genai.configure(api_key=os.environ["GEMINI_API_KEY"])

async def call_ai(
    prompt: str,
    system: str,
    task: str = "general", # "fast" uses 8b, else 70b
    stream: bool = False
) -> str:
    model = "llama-3.1-8b-instant" if task == "fast" else "llama-3.3-70b-versatile"

    try:
        response = await groq_client.chat.completions.create(
            model=model,
            messages=[
                {"role": "system", "content": system},
                {"role": "user", "content": prompt}
            ],
            stream=stream
        )
        if stream:
            return response # return stream object
        return response.choices[0].message.content

    except groq.RateLimitError:
        # Fall back to Gemini
        gemini_model = genai.GenerativeModel(
            model_name="gemini-1.5-flash",
            system_instruction=system
        )
        response = await gemini_model.generate_content_async(prompt)
        return response.text

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"AI routing failed: {str(e)}")

```

Task → model assignment

Task	Model	Reason
Resume parsing (extract structure)	llama-3.1-8b-instant	Fast, structured output, low complexity
Resume scoring	llama-3.1-8b-instant	Scoring rubric is deterministic, 8b handles it
Weakness detection	llama-3.1-8b-instant	Pattern matching task
Resume rewriting	llama-3.3-70b-versatile	Highest quality needed, this is the money feature
Role alignment	llama-3.3-70b-versatile	Nuanced judgment required
Internship fit explanations	llama-3.1-8b-instant	Short outputs, fast is better
AI Copilot chat	llama-3.3-70b-versatile	Conversational quality matters
Strategy reports	llama-3.3-70b-versatile	Personalized, nuanced
Follow-up email templates	llama-3.1-8b-instant	Template generation, fast

6. Stripe Integration — Full Implementation

Setup

```
bash
pip install stripe # FastAPI
npm install @stripe/stripe-js # Next.js
```

Env vars:

```
STRIPE_SECRET_KEY=sk_live_...  
STRIPE_WEBHOOK_SECRET=whsec_...  
STRIPE_PRICE_ID=price_... # your $5.99/month price ID  
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live_...
```

Flow: Free user → Pro subscriber

1. User clicks "Upgrade to Pro" anywhere in app
2. Next.js calls server action → `POST /api/stripe/create-checkout-session`
3. FastAPI creates Stripe Checkout Session with `price_id` and `customer_email`
4. Returns `session.url` → Next.js redirects user to Stripe-hosted checkout
5. User completes payment on Stripe
6. Stripe fires `customer.subscription.created` webhook to `POST /api/stripe/webhook`
7. FastAPI verifies webhook signature, extracts `customer.email`
8. FastAPI calls Convex mutation `updateUserProStatus(email, isPro: true)`
9. User is redirected to `/dashboard?upgraded=true` with success toast

FastAPI Stripe endpoints

python

```

# stripe_router.py

@router.post("/create-checkout-session")
async def create_checkout_session(user_email: str):
    session = stripe.checkout.Session.create(
        payment_method_types=["card"],
        mode="subscription",
        line_items=[{"price": os.environ["STRIPE_PRICE_ID"], "quantity": 1}],
        customer_email=user_email,
        success_url="https://internsight.app/dashboard?upgraded=true",
        cancel_url="https://internsight.app/pricing",
    )
    return {"url": session.url}

@router.post("/webhook")
async def stripe_webhook(request: Request):
    payload = await request.body()
    sig_header = request.headers.get("stripe-signature")

    try:
        event = stripe.Webhook.construct_event(
            payload, sig_header, os.environ["STRIPE_WEBHOOK_SECRET"]
        )
    except stripe.error.SignatureVerificationError:
        raise HTTPException(status_code=400, detail="Invalid signature")

    if event["type"] == "customer.subscription.created":
        customer_email = event["data"]["object"]["customer_email"]
        await convex_client.mutation("users:setProStatus", {
            "email": customer_email, "isPro": True
        })

    elif event["type"] in ["customer.subscription.deleted", "customer.subscription.p
        customer_email = event["data"]["object"]["customer_email"]
        await convex_client.mutation("users:setProStatus", {
            "email": customer_email, "isPro": False
        })

    return {"received": True}

@router.post("/create-portal-session")
async def create_portal_session(stripe_customer_id: str):
    # Lets users manage/cancel subscription

```

```
session = stripe.billing_portal.Session.create(
    customer=stripe_customer_id,
    return_url="https://internsight.app/settings"
)
return {"url": session.url}
```

Pro gate in Next.js

typescript

```
// lib/gates.ts
export function requirePro(user: User) {
    if (!user.isPro) {
        redirect('/pricing?reason=pro-required')
    }
}

// In any server component or server action:
const user = await getCurrentUser()
requirePro(user)
```

Pro gate in FastAPI

python

```
# middleware
async def require_pro(user_id: str = Depends(get_current_user)):
    user = await convex_client.query("users:getById", {"id": user_id})
    if not user.get("isPro"):
        raise HTTPException(status_code=403, detail="Pro subscription required")
    return user
```

7. Convex Database Schema

typescript

```

// convex/schema.ts
import { defineSchema, defineTable } from "convex/server";
import { v } from "convex/values";

export default defineSchema({
  users: defineTable({
    clerkId: v.string(),
    email: v.string(),
    name: v.string(),
    school: v.optional(v.string()),
    gradYear: v.optional(v.number()),
    gpa: v.optional(v.number()),
    targetRole: v.optional(v.string()),
    isPro: v.boolean(),
    stripeCustomerId: v.optional(v.string()),
    stripeSubscriptionId: v.optional(v.string()),
    onboardingComplete: v.boolean(),
    createdAt: v.number(),
  }).index("by_clerk_id", ["clerkId"]).index("by_email", ["email"]),

  resumes: defineTable({
    userId: v.id("users"),
    storageId: v.string(), // Convex file storage ID
    filename: v.string(),
    uploadedAt: v.number(),
    parsed: v.optional(v.any()), // structured resume object
    currentVersion: v.boolean(),
  }).index("by_user", ["userId"]),

  analyses: defineTable({
    userId: v.id("users"),
    resumeId: v.id("resumes"),
    score: v.number(), // 0-100
    breakdown: v.object({
      experience: v.number(),
      metrics: v.number(),
      structure: v.number(),
      keywords: v.number(),
    }),
    weaknesses: v.array(v.object({
      type: v.string(),
      severity: v.string(), // "critical" | "warning" | "suggestion"
      message: v.string(),
    })),
  })
});

```

```

    suggestion: v.string(),
  })),
  rewrittenResume: v.optional(v.string()), // markdown string
  targetRole: v.string(),
  createdAt: v.number(),
}).index("by_user", ["userId"]).index("by_resume", ["resumeId"]),

alignments: defineTable({
  userId: v.id("users"),
  resumeId: v.id("resumes"),
  role: v.string(),
  status: v.string(), // "READY" | "CLOSE" | "NOT_READY"
  matchPct: v.number(),
  gaps: v.array(v.string()),
  reasoning: v.string(),
  estimatedWeeks: v.optional(v.number()),
  createdAt: v.number(),
}).index("by_user", ["userId"]),

internships: defineTable({
  title: v.string(),
  company: v.string(),
  location: v.string(),
  remote: v.boolean(),
  skills: v.array(v.string()),
  url: v.string(),
  postedAt: v.number(),
  source: v.string(),
  expiresAt: v.number(), // TTL for cache invalidation
}),

applications: defineTable({
  userId: v.id("users"),
  company: v.string(),
  role: v.string(),
  url: v.optional(v.string()),
  status: v.string(), // kanban column
  appliedAt: v.optional(v.number()),
  followUpSentAt: v.optional(v.number()),
  notes: v.optional(v.string()),
  matchScore: v.optional(v.number()),
  createdAt: v.number(),
}).index("by_user", ["userId"]),

```

```

conversations: defineTable({
  userId: v.id("users"),
  messages: v.array(v.object({
    role: v.string(),          // "user" | "assistant"
    content: v.string(),
    createdAt: v.number(),
  })),
  updatedAt: v.number(),
}).index("by_user", ["userId"]),
});

```

8. FastAPI Project Structure

```

packages/api/
├─ main.py                # FastAPI app init, CORS, middleware
├─ requirements.txt
├─ Dockerfile
├─ .env
├─ core/
│   ├─ ai_router.py       # Groq + Gemini routing logic
│   ├─ convex_client.py   # HTTP client to Convex mutations/queries
│   └─ auth.py            # Clerk JWT verification middleware
├─ routes/
│   ├─ resume.py          # /api/resume/* endpoints
│   ├─ roles.py           # /api/roles/* endpoints
│   ├─ internships.py     # /api/internships/* endpoints
│   ├─ copilot.py         # /api/copilot/* endpoints (streaming)
│   ├─ strategy.py        # /api/strategy/* endpoints
│   └─ stripe.py          # /api/stripe/* endpoints
├─ services/
│   ├─ resume_parser.py   # PDF/DOCX text extraction
│   ├─ resume_scorer.py   # Scoring logic + weakness detection
│   ├─ resume_rewriter.py # Jake-format rewrite prompts
│   ├─ role_aligner.py    # Role expectation schemas + alignment
│   ├─ internship_matcher.py # Fit scoring logic
│   └─ strategy_engine.py # Weekly report generation
├─ data/
│   └─ roles.json         # Role expectation definitions
└─ prompts/
    ├─ parse_prompt.txt   # Resume extraction system prompt
    └─ score_prompt.txt   # Scoring rubric system prompt

```

```
|— rewrite_prompt.txt      # Jake-format rewrite system prompt
|— align_prompt.txt        # Role alignment evaluation prompt
|— copilot_system.txt      # Copilot base system prompt
```

9. Next.js Project Structure

```
packages/web/
|— app/
|   |— layout.tsx          # Root layout with ClerkProvider + ThemeProvider
|   |— page.tsx            # Landing page (public)
|   |— (auth)/
|       |— sign-in/        # Clerk sign-in page
|       |— sign-up/        # Clerk sign-up page
|   |— onboarding/
|       |— page.tsx        # New user onboarding form
|   |— dashboard/
|       |— page.tsx        # Main dashboard with stat cards
|   |— resume/
|       |— page.tsx        # Upload page
|       |— analysis/page.tsx # Score + results view
|       |— builder/page.tsx # Jake-style editor
|   |— roles/
|       |— page.tsx        # Role alignment overview
|       |— [roleId]/page.tsx # Detailed role breakdown
|   |— explore/
|       |— page.tsx        # Internship discovery
|   |— copilot/
|       |— page.tsx        # AI chat interface
|   |— tracker/
|       |— page.tsx        # Application kanban
|   |— pricing/
|       |— page.tsx        # Pricing page with Stripe CTA
|   |— settings/
|       |— page.tsx        # Profile + subscription management
|— components/
|   |— ui/                 # shadcn/ui components
|   |— resume/             # Resume-specific components
|   |— roles/              # Role alignment components
|   |— copilot/            # Chat UI components
|   |— tracker/            # Kanban board components
|   |— shared/             # Nav, sidebar, modals, toasts
```

```
├─ lib/
│   ├─ api.ts           # FastAPI client (typed fetch wrappers)
│   ├─ gates.ts         # Pro subscription gate helpers
│   ├─ stripe.ts        # Stripe client helpers
│   └─ utils.ts         # shadcn/ui utils
├─ convex/
│   ├─ schema.ts        # Database schema (see Section 7)
│   ├─ users.ts         # User mutations + queries
│   ├─ resumes.ts       # Resume mutations + queries
│   ├─ analyses.ts      # Analysis queries
│   ├─ applications.ts  # Application tracker CRUD
│   ├─ internships.ts   # Internship cache queries
│   ├─ crons.ts         # Scheduled functions (follow-ups, listing refresh)
│   └─ _generated/      # Auto-generated Convex types
├─ middleware.ts        # Clerk route protection
└─ public/
```

10. Environment Variables

Next.js (.env.local)

```
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=
CLERK_SECRET_KEY=
CLERK_WEBHOOK_SECRET=
NEXT_PUBLIC_CONVEX_URL=
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=
NEXT_PUBLIC_API_URL=https://your-railway-app.railway.app
```

FastAPI (.env)

```
GROQ_API_KEY=
GEMINI_API_KEY=
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
STRIPE_PRICE_ID=
CONVEX_URL=
CONVEX_DEPLOY_KEY=
CLERK_SECRET_KEY=
RESEND_API_KEY=
```

11. Key Product Decisions & Rationale

Why Groq + Gemini instead of OpenAI/Claude? Groq's free tier (14,400 req/day on llama-3.3-70b) covers development and early users at zero cost. Gemini Flash is the fallback because it's also free (1,500 req/day) and fast. Both produce quality output for resume tasks. When revenue justifies it, upgrade the 70b tasks to a higher-quality model. Never pay for AI before you have paying users.

Why \$5.99/month (not \$9.99, not freemium-heavy)? Low enough that a broke college student will pay it. High enough to be taken seriously. The price point removes the "is this worth it?" friction. One accepted internship offer (even at \$20/hr × 12 weeks = \$9,600) makes \$5.99 feel free. That's the mental model to sell.

Why Convex instead of Supabase/PlanetScale? Real-time subscriptions are built in — no Pusher, no WebSockets to configure. The resume builder needs live score updates. The tracker needs real-time sync. Convex does both without infrastructure overhead. The dev experience is also significantly faster for solo developers.

Why one flat price instead of tiers? Students don't want to think about what features they're getting. "Unlimited everything for \$5.99" is a complete sentence. Tier complexity adds decision friction and support complexity. Add tiers when you have enterprise customers, not before.

Why Jake's Resume format specifically? It's the most widely recognized, ATS-optimized format in tech recruiting. Thousands of successful tech candidates have used it. Enforcing it gives InternSight a specific, defensible output standard — "we produce Jake-format resumes" is a concrete claim. Generic "improved" resumes are not.

Why FastAPI instead of Next.js API routes for AI? Python has the best AI ecosystem. pdfplumber, python-docx, and the Groq/Gemini SDKs are all Python-native. Running AI in Next.js API routes means JavaScript SDKs with worse tooling and no ability to use Python-specific parsing libraries. The separation also lets you scale the AI layer independently.

12. Development Phase Summary

Phase	Range	Focus
0	0-5%	Monorepo scaffolding, Next.js + FastAPI + Convex init
1	5-15%	Clerk auth, dashboard shell, onboarding
2	15-35%	Resume Intelligence Engine (parse, score, rewrite)
3	35-45%	Resume Builder (live editor, Jake enforcement, PDF export)

Phase	Range	Focus
4	45-60%	Role Alignment Engine
5	60-70%	Internship Discovery + fit scoring
6	70-80%	AI Copilot (streaming chat)
7	80-90%	Application Strategy Engine + tracker
8	90-100%	Stripe integration, polish, Vercel + Railway deployment

Critical path: Phase 2 → Phase 4 → Phase 6. If those three work well, you have a product. Everything else adds retention and monetization.

13. AI Prompt Philosophy

Every prompt follows this structure:

```
[ROLE DEFINITION]
You are InternSight's [specific engine]. Your only job is [specific task].
Never do [anti-patterns]. Always [key constraints].

[CONTEXT INJECTION]
Target role: {target_role}
Student background: {parsed_summary}
Current score: {score}/100

[TASK DEFINITION]
[Specific instruction with output format]

[OUTPUT FORMAT]
Return ONLY valid JSON matching this schema: {schema}
No explanation. No preamble. No markdown fences.
```

Never use vague prompts like "improve this resume." Always provide the role, the rubric, and the exact output format expected. JSON output is enforced everywhere except the copilot.

This document is the single source of truth for InternSight. Update it when major architectural decisions change. Paste it as context at the start of any AI coding session.